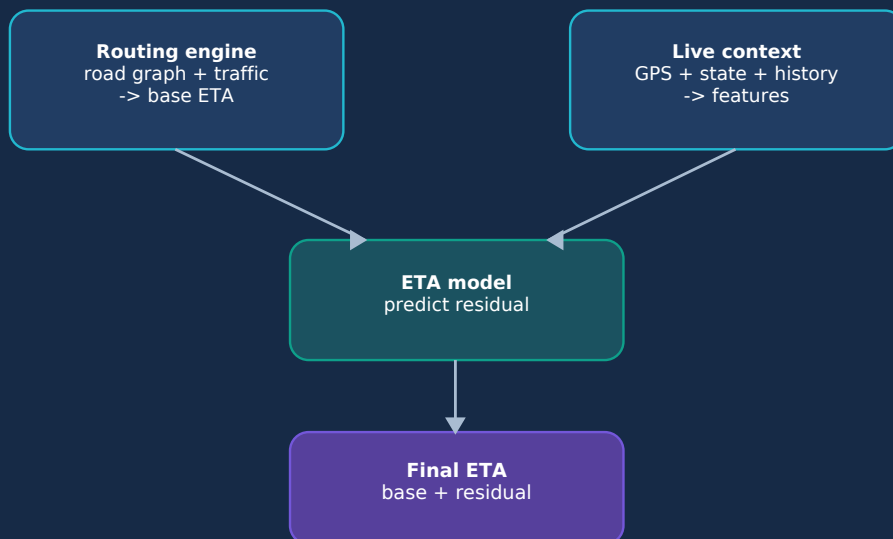


DAY 02 / 100

Design an Uber-Style ETA Prediction System

Rider pickup, assigned-driver, trip, and remaining ETA

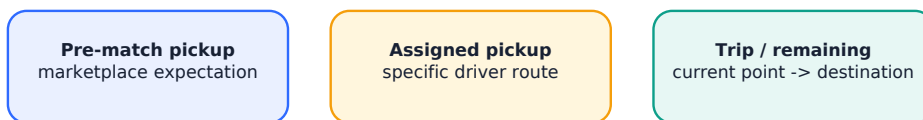


A state-aware, routing-grounded, continuously updated ML system.

PUBLIC REVISION NOTES | 18-PAGE CONCISE EDITION

Define the ETA surface before choosing a model

ETA is time remaining from a prediction timestamp to a state-specific endpoint. A production system predicts several related but different quantities.



State determines the endpoint, label, features, refresh policy, and error cost.

Surface	Start and endpoint	Main uncertainty
Pre-match pickup ETA	Request now -> pickup by an as-yet-unassigned driver	Dispatch/matching delay plus which driver will accept
Assigned pickup ETA	Current driver state -> arrival at rider pickup	Driver route choice, GPS, traffic, pickup access
Trip ETA	Pickup/trip start -> destination	Route, traffic, incidents, destination access
Remaining ETA	Any active state now -> current endpoint	Progress, reroute, live conditions, event ordering

Do not train one vague 'ETA' label. Each surface needs explicit start/end timestamps, state, eligible features, refresh triggers, and business error costs.

Product goals

Increase rider trust, reduce pickup coordination friction and cancellations, improve dispatch and planning, and avoid both optimistic underprediction and excessive pessimism. ETA stability and latency are part of product quality, not secondary engineering metrics.

Requirements, assumptions, and state transitions

Dimension	Requirement / design assumption
Scale	Very high request volume; multiple refreshes per trip. No unsupported claim about a named platform's current QPS.
Latency	Model correction should add only a few milliseconds; end-to-end routing, features, model, and post-processing need a P99 budget.
Freshness	Driver GPS and state in seconds; traffic/incidents in seconds to minutes; history slower.
Availability	Routing-only ETA is the primary safe fallback when ML or dynamic features fail.
Consistency	A newer state/version must never be overwritten by a late event or stale prediction.
Updates	Refresh on state, route, progress, traffic, or elapsed-time thresholds rather than every heartbeat.
Generalization	New roads/cities/routes back off to structural, coarse-geographic, and routing features.

State transitions

- Requested -> matching -> assigned.
- Driver en route -> arrived at pickup.
- Rider onboard -> active trip.
- Arrived -> completed or cancelled.
- Endpoint changes at pickup; route progress resets after reroute.

State invariants

- Sequence/version must be monotonic.
- Endpoint must match the current state.
- Prediction timestamp precedes endpoint.
- Final ETA is non-negative and bounded.
- Feature freshness and route version travel with the prediction.

Pre-match pickup ETA combines marketplace uncertainty with travel time. After assignment, the prediction conditions on a specific driver and route; the two labels should not be silently mixed.

End-to-end online ETA architecture

The routing engine owns network structure and route feasibility. ML corrects systematic real-world error using current context.

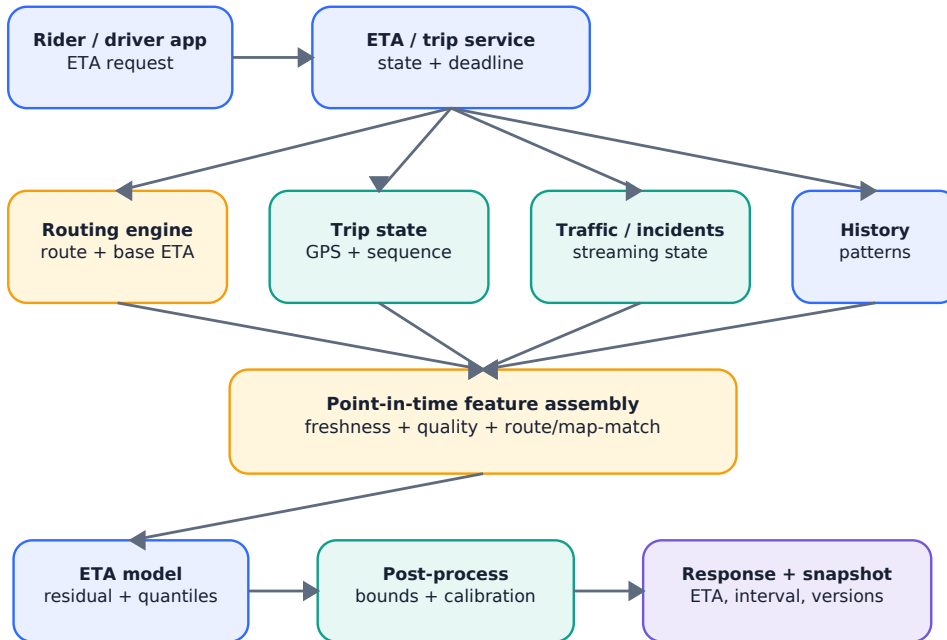
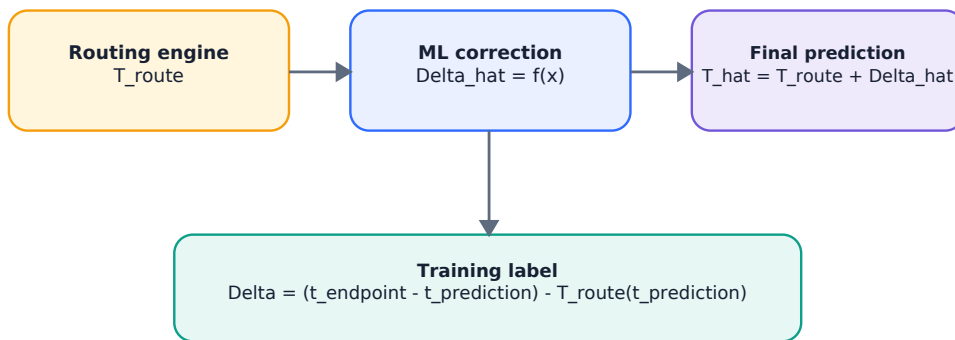


Figure 1. The online path is deadline-aware; prediction snapshots feed delayed-label training.

Boundary	Contract	Failure behavior
Trip service	trip_id, surface/state, endpoint, event version, request deadline	Reject stale state; use anonymous/pre-match path when assignment is unknown
Routing	route polyline/version, distance, segment data, base ETA	Reuse recent safe route or degrade transparently
Features	event-time values with freshness and quality	Default/drop stale optional features; fail on corrupt state
Model	residual plus optional quantiles and model version	Use reduced-feature or routing-only path
Response	final ETA, display interval, prediction ID, versions	Never return impossible/unsafe values

Routing plus ML residual learning



Negative residuals are valid; only the final ETA must be physically valid.

Figure 2. The model learns post-processing error around a physically grounded routing estimate.

Why this decomposition works

- The routing graph already captures topology, legal movement, and candidate paths.
- ML solves a smaller problem: systematic error from traffic, time, location, incidents, request type, and behavior.
- ML value is measured directly as lift over routing-only ETA.
- Road-network changes remain the routing system's responsibility.
- If ML fails, the system still has a coherent baseline.

Approach	Benefit	Risk
Routing only	Deterministic, interpretable, robust	Misses systematic context and real-world deviation
Pure ML total ETA	Simple output contract	Must relearn route structure; harder debugging and fallback
Routing + residual ML	Strong inductive bias and clean baseline	Requires exact route/snapshot alignment and residual labels
Segment-level prediction	Fine local modeling	Aggregation, route consistency, and label complexity

Uber's DeepETA article explicitly describes this hybrid ETA post-processing formulation [1].

Feature engineering and freshness contracts

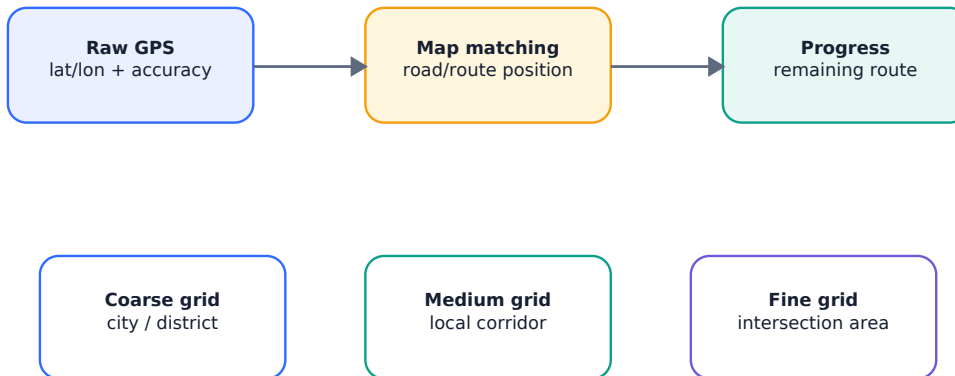
Family	Examples	Freshness / quality
Routing	base ETA, remaining distance, road classes, intersections, turns, route version	Per route; invalidate after reroute
Spatial	origin/destination/current point, corridor, zones, road segment, grid IDs	GPS accuracy + map-match confidence
Traffic	current/historical speed ratio, congestion, variance, segment delay	Seconds/minutes; source timestamp
Temporal	minute/hour, weekday, holiday, rush period, season	Request-time
Incidents/weather	closure, crash, construction, event, rain, visibility	Near-real-time; coverage indicator
Trip/marketplace	surface, state, vehicle, city, dispatch delay, candidate supply	State/request dependent
Progress	time since request/pickup, off-route distance, remaining fraction, stop duration	Seconds; monotonic where expected
History	segment/zone travel stats, seasonal averages, route deviation patterns	Minutes to hours

Transformation

- Embeddings for high-cardinality road/zone/city IDs.
- Raw normalized continuous values plus selected buckets.
- Cyclic/time buckets and holiday indicators.
- Explicit missingness, staleness, and quality features.
- Quantile buckets where density is highly uneven.
- Interactions such as rain x rush hour x road class.
- Shared feature definitions for online and training paths.
- Event-time joins; never use values observed after prediction.

Freshness is a model input. A traffic speed without source time, coverage, or quality can be more harmful than a known fallback.

Map matching and geospatial representation



Combine structural road features with multi-resolution embeddings; IDs alone are brittle.

Figure 3. Convert noisy GPS into route progress before using it as remaining-ETA context.

Map matching gap addressed

Raw GPS may jump across roads, arrive out of order, or drift near flyovers and dense streets. Match the observation to a plausible road/route position using location accuracy, heading, speed, prior position, and route continuity. Emit map-match confidence and off-route distance; low confidence triggers uncertainty or fallback rather than false precision.

Multi-resolution location features

- Coarse cells capture city and district context; fine cells capture local corridor/intersection behavior.
- Feature hashing bounds embedding memory; multiple independent hashes reduce collision damage.
- Back off from fine to coarse grids for sparse/new locations.
- Combine learned IDs with road attributes and routing features so cold start is not pure memorization.

DeepETA documents multi-resolution grids, hashing, and geospatial embeddings as an accuracy/latency/memory trade-off [1].

Model architecture: baseline first, complexity earned

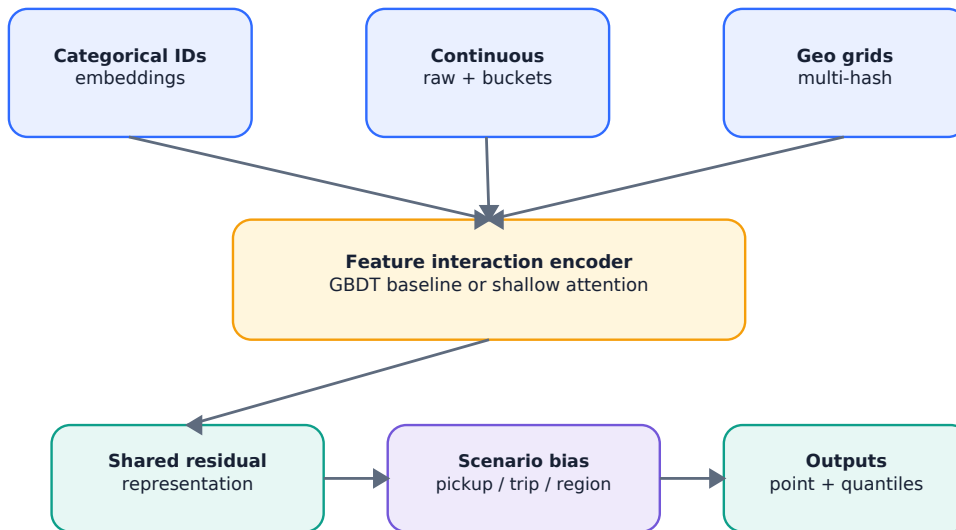


Figure 4. A shared representation can support pickup/trip/region adjustment without maintaining many unrelated models.

Stage	Model	Why
V0	Routing-only	Establish correctness, latency, and a non-ML baseline.
V1	LightGBM/XGBoost residual	Strong tabular baseline; fast, debuggable, low operational complexity.
V2	Embeddings + shallow MLP	Scales high-cardinality spatial/context features.
V3	Feature-token attention	Learns pairwise interactions across spatial, temporal, traffic, and request features.
V4	Linear-attention variant	Reduces interaction cost when K feature tokens exceed embedding dimension d.
V5	Shared model + segment bias	Global sharing with lightweight pickup/trip/region specialization.

DeepETA reports moving from a large XGBoost post-processor to a low-latency deep model with self-attention, discretized/embedded inputs, and segment bias adjustment [1]. That is documented inspiration, not a requirement for every ETA implementation.

Sparse parameters, feature interactions, and latency

Sparse parameters

Large road/grid embedding tables may contain most model parameters, but a request touches only selected rows. Parameter count is not active inference compute.

Useful optimization

- O(1)-style hash/embedding lookup.
- Shallow dense layers.
- Quantization/distillation after accuracy checks.
- Bounded feature-token count.

Attention choice

For K feature tokens of width d , standard self-attention is approximately $O(K^2 \cdot d)$. A linear formulation can be approximately $O(K \cdot d^2)$ for the relevant implementation, useful when $K > d$.

Operational caution

- Validate actual kernels and batch sizes.
- Do not add positional encoding when feature order has no meaning.
- Tree ensembles may deliver most of the gain more simply.
- Optimize measured P99, not Big-O alone.

Optimization	Benefit	Trade-off
Route reuse/cache	Avoid repeated route computation	Stale after progress/reroute; strict invalidation
Stable-feature cache	Reduces feature assembly cost	Key/TTL/version complexity
Sparse embedding lookup	Large capacity with low active compute	Memory, cold start, distributed lookup
Micro-batching	Higher hardware utilization	Queueing may hurt interactive tail latency
Model compression	Lower memory and compute	Accuracy/calibration degradation
Event-driven refresh	Avoids redundant inference	Needs carefully tuned triggers

The target is the simplest model that improves routing-only accuracy within the complete end-to-end latency and cost budget.

Loss functions, asymmetry, and uncertainty

Let e = predicted ETA - actual ETA. Underprediction means $e < 0$.

Objective	Use	Caution
MAE	Directly minimizes typical absolute minutes of error	Non-smooth; treats early/late equally
Huber	Quadratic near zero, linear for large errors; robust default	delta must match ETA scale/segments
Asymmetric Huber	Weights underprediction and overprediction differently	Weights encode product policy and need experimentation
Quantile / pinball	P50/P80/P90 outputs and uncertainty intervals	Quantiles can cross and require coverage calibration

Asymmetric Huber

$L(e) = w_{\text{under}} * \text{Huber}(e)$ when $e < 0$; $w_{\text{over}} * \text{Huber}(e)$ when $e \geq 0$

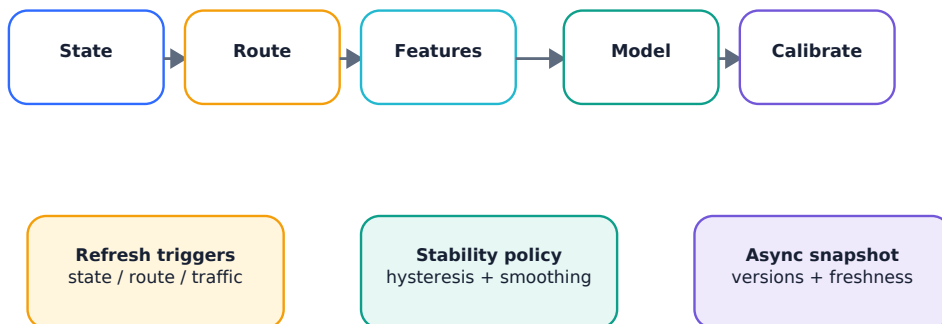
A late-looking ETA can cause missed expectations; an overly conservative ETA can reduce conversion. The relative cost is product- and surface-specific: pre-match pickup, assigned pickup, fare calculation, and trip arrival may use different point estimates.

Uncertainty output

- Return a point ETA plus calibrated P80/P90 interval where the product can use it.
- Measure empirical interval coverage and width by city, horizon, traffic, and GPS quality.
- Represent uncertainty from stale traffic, low-confidence map matching, pre-match assignment, and unusual routes.
- Clamp the final ETA, not the residual; a negative residual can be correct.

DeepETA documents parameterized asymmetric Huber loss for different ETA point-estimate requirements [1].

Online flow, refresh triggers, and displayed stability



Do not recompute on every GPS heartbeat; respond to meaningful change and deadlines.

Figure 5. Separate prediction freshness from display stability; genuine route/traffic changes must still surface quickly.

Refresh when something meaningful changes

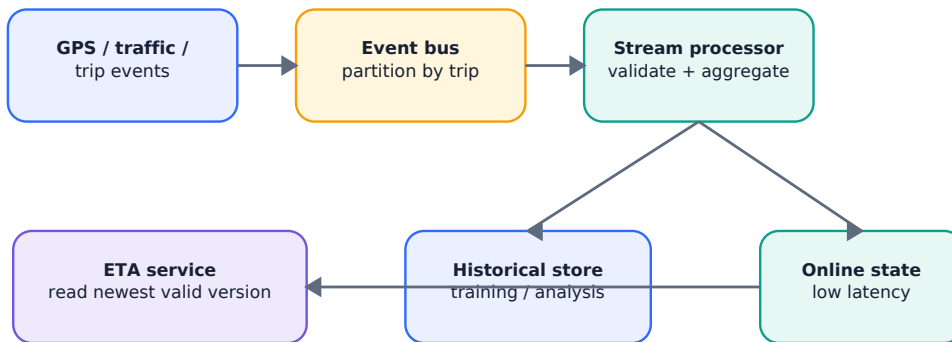
Trigger	Example	Action
State	driver assigned, arrived, rider onboard	Change endpoint/features; recompute immediately
Route	reroute, off-route, route version change	Invalidate base ETA/cache; map-match and recompute
Progress	distance/time threshold crossed	Update remaining-route features
Traffic/incident	material corridor delay change	Refresh impacted trips only
Time	maximum age reached	Safety refresh if no other event fired

ETA stability gap addressed

Raw model updates can oscillate. Use small-change hysteresis, confidence-aware smoothing, and minimum display intervals, but bypass smoothing after a real reroute, state transition, or large incident. Log both raw and displayed ETA. Evaluate update-to-update volatility, maximum jump, and time taken to reflect a genuine change.

Never make the ETA look stable by hiding reality. Stability policy should remove noise, not delay material corrections.

Streaming, caching, and event ordering



timestamp + sequence + idempotency + monotonic state validation

Figure 6. Streaming and low-latency state are justified by live GPS, traffic, and trip-state freshness.

Late-event example

A: DRIVER_EN_ROUTE, event_time=100, seq=14
 B: ARRIVED_PICKUP, event_time=105, seq=15
 A arrives after B -> ignore A for current state

Consistency controls

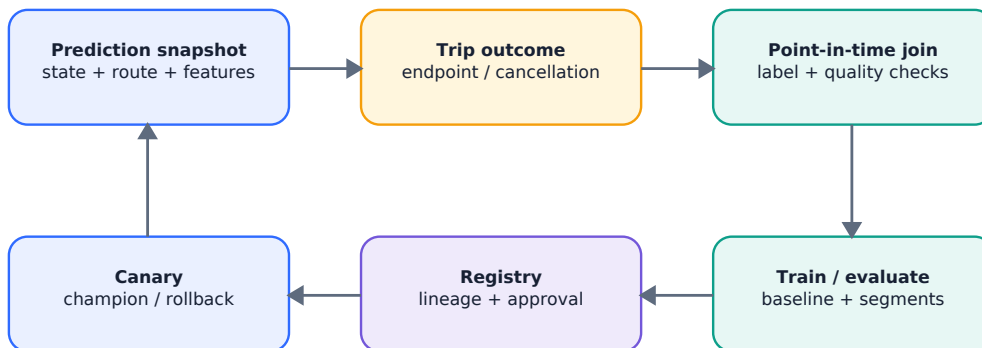
- Partition trip events by trip_id.
- Event timestamp + monotonic sequence.
- Idempotent writes and deduplication.
- State-transition validation.
- Compare-and-set for online trip state.

Cache controls

- Route cache keyed by route/version/state.
- TTL plus event-driven invalidation.
- Stable/history features cached longer.
- Never cache the final ETA blindly.
- Reapply freshness and state checks on read.

Kafka-like streaming and Redis-like online state are design choices, not ceremonial components. Use them only where freshness, ordering, and latency require them.

Prediction logging, delayed labels, and retraining



Labels arrive after the endpoint; cancellation creates censoring and selection bias.

Figure 7. A prediction must be reproducible before its delayed endpoint label can be trusted.

Snapshot contract

prediction_id, trip_id, surface/state, prediction_time, endpoint definition, state/event version, route/version, base_eta, residual, final_eta, quantiles, model_version, feature references/values, freshness/quality, raw_displayed_eta

Delayed and censored outcomes

- Completed trips yield endpoint timestamps and residual labels.
- Cancelled trips may be right-censored: the true pickup/destination time is unobserved.
- Dropping all cancellations can bias training toward easier successful trips; retain censoring reason and evaluate selection by segment.
- Join only features known at prediction time; route and traffic revisions after the fact are leakage.

Retraining cadence follows mature labels, volume, drift, and degradation - not a universal schedule. The Michelangelo reference establishes the end-to-end platform pattern of data management, training, evaluation, deployment, prediction, and monitoring [2].

Model, product, stability, and system metrics

Layer	Metrics	Question
Point accuracy	MAE, Median AE, RMSE	How large is typical and squared error?
Tail	P90/P95/P99 absolute error	How bad are the worst user-visible misses?
Bias	Mean signed error, underprediction rate	Are ETAs systematically optimistic/pessimistic?
Uncertainty	P80/P90 coverage, interval width, quantile crossing	Do intervals mean what they claim?
Baseline	Delta MAE/tail vs routing-only	What value does ML add?
Stability	Update volatility, max jump, late-correction delay	Is the displayed ETA trustworthy over time?
Product	Cancellation, conversion, pickup friction, satisfaction	Does accuracy improve outcomes?
System	P50/P95/P99 latency, errors, fallback, cost	Can it serve reliably at peak?

Required slices

- Pre-match vs assigned pickup vs trip.
- City/region and new vs mature market.
- Rush vs off-peak; normal vs incident/weather.
- Short vs long horizon and remaining distance.
- High vs low GPS/map-match confidence.
- New/sparse roads and rerouted trips.
- Vehicle/trip type and model version.
- Fresh vs stale traffic and fallback traffic.

Avoid MAPE as the primary metric: percentage error becomes unstable when remaining ETA is close to zero. Use absolute-minute errors and horizon-aware slices.

A model can improve global MAE while worsening underprediction, tail error, update stability, or a critical city. Launch decisions need the full scorecard.

Monitoring, failure modes, and graceful degradation

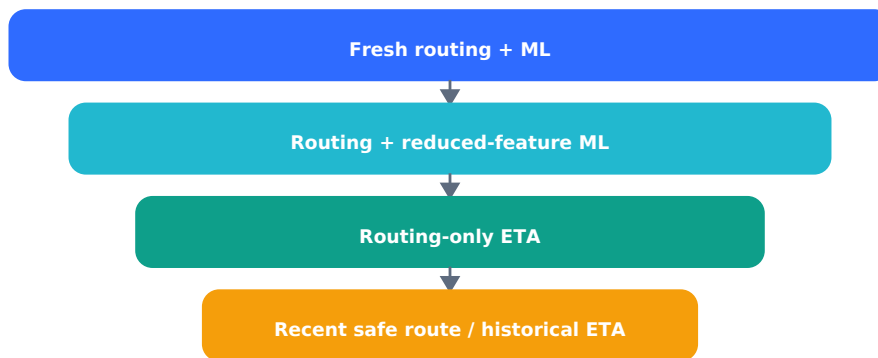


Figure 8. Quality degrades stepwise; impossible or stale ETAs never overwrite valid state.

Failure / drift	Detection	Response
Traffic stale	source age, coverage, speed distribution	Drop/down-weight; use historical traffic
GPS/map match weak	accuracy, jump, confidence, off-route	Last valid state + wider interval or route fallback
Routing unhealthy	latency, failures, route-change anomalies	Recent safe route if valid; degrade/withhold precision
Model unhealthy	P99/error/fallback/model canary	Last-known-good or routing-only
State race	sequence regression/invalid transition	Reject stale event/prediction
Prediction drift	bias, tail error, segment degradation	Rollback, retrain, or segment safeguard
Display instability	jump/volatility metrics	Tune smoothing; inspect route/traffic triggers

Operational signals

Monitor routing/model/feature latency separately, state age, feature freshness, route-cache hit/invalidation, fallback rate, prediction volume, resource saturation, and cost per prediction. Every alert needs an owner, runbook, and rollback target.

Alternatives, trade-offs, and evolution path

Decision	Option A	Option B	Decision rule
Target	Total ETA	Routing residual	Prefer residual with a strong routing baseline and exact labels
Model	GBDT	Deep/attention	Move only for measured segment lift within P99/cost
Geography	Raw/structural	Grid embeddings	Combine for generalization plus local accuracy
Specialization	Separate models	Shared + bias/head	Balance segment gain against operational fragmentation
Output	Point ETA	Quantiles	Add quantiles only when product uses and calibration supports them
Refresh	Fixed interval	Event/threshold	Event-driven for scale; retain maximum-age safety trigger
Stability	Raw updates	Smoothing/hysteresis	Suppress noise without hiding real changes

Evolution

V0 routing-only -> V1 GBDT residual -> V2 spatial/high-cardinality embeddings -> V3 shallow interaction model -> V4 low-latency attention -> V5 shared global model with segment adjustment -> V6 calibrated quantiles and advanced censored-label handling

Complexity gates

- Do not add a deep model before point-in-time logging and routing baseline evaluation are trustworthy.
- Do not increase refresh frequency without measuring accuracy gain, volatility, and cost.
- Do not split models by every city before a shared model's failure modes are quantified.
- Do not claim uncertainty until empirical coverage is calibrated by surface and horizon.

The architect's job is not to name the most advanced model. It is to assign responsibilities correctly across routing, ML, streaming state, policy, and fallbacks.

Mental model and design checklist

Step	Question
STATE	What phase are we in, and what endpoint defines remaining time?
ROUTE	What path/base ETA does the routing engine provide?
MATCH	Where is the vehicle on the road/route, and how confident are we?
FEATURE	Which live and historical signals explain routing error?
PREDICT	What residual and uncertainty does the model estimate?
POST-PROCESS	Are final ETA, interval, and displayed update valid and calibrated?
LOG	Can we reproduce the state, route, features, and versions later?
LEARN	How do delayed/censored outcomes become point-in-time-correct labels?
OPERATE	What is the P99 budget, refresh policy, and fallback ladder?

Problem and ML

- Separate pre-match, assigned pickup, and trip ETA.
- Define start/end timestamps and residual label.
- Explain routing + ML responsibilities.
- Prioritize spatial, traffic, route, state, and freshness features.
- Start with GBDT; justify deeper interaction models.
- Choose Huber/asymmetry/quantiles from product cost.

System and evaluation

- Map-match noisy GPS and track confidence.
- Refresh on meaningful events; control display volatility.
- Version trip state and reject late updates.
- Log prediction snapshots for delayed labels.
- Measure tail, bias, coverage, stability, business, latency.
- Fallback to routing-only ETA.

One-line takeaway: ETA is a state-dependent, continuously updated residual prediction grounded by routing and kept honest by event-time data, calibration, stability controls, and fallbacks.

Approved references and public-use note

[1] Uber Engineering. **DeepETA: How Uber Predicts Arrival Times Using Deep Learning**. February 10, 2022.
uber.com/blog/deepeta-how-uber-predicts-arrival-times

[2] Uber Engineering. **Meet Michelangelo: Uber's Machine Learning Platform**. September 5, 2017.
uber.com/in/en/blog/michelangelo-machine-learning-platform

Reference policy: these are the only external references carried forward from the prior Day 2 session. No transcript URL was recoverable, so none was added.

Public scope

This document is an educational Uber-style system design. Statements explicitly attributed to DeepETA or Michelangelo come from the references above. Candidate counts, latency budgets, thresholds, and proposed additions are design choices - not claims about Uber's current internal implementation.

What was refined

The original 15-page draft's core design was preserved and reorganized. Gaps were addressed for pre-match versus assigned pickup ETA, GPS map matching, route uncertainty, display stability, censored cancellation labels, point-in-time joins, interval coverage, and explicit state/version contracts. Repetition was reduced and production trade-offs were made easier to revise.

END OF DAY 02

**Route the structure. Learn the residual.
Update only when the world changes.**